

## 2.6 How to Decide Which Algorithm to Use

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔

CS XI

Chapter 2: Computational Thinking &amp; Algorithm

Lecture 13 of 13

## 2.6 How to Decide Which Algorithm to Use

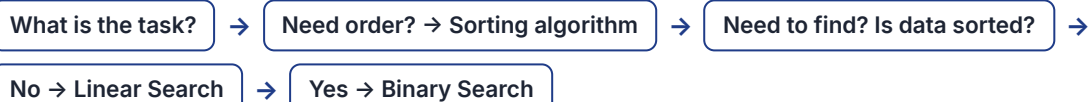
### LEARN IT

Before choosing an algorithm, ask three questions:

- **What is the input size?** (10 items or 10 million?)
- **Is the data sorted?**
- **Do we need speed or memory efficiency?**

**Simple decision guide:**

- If searching on **unsorted** data → use **Linear Search**.
- If searching on **sorted** data → use **Binary Search**.
- If data needs **ordering** → use a **sorting** algorithm.



### Comparison of the four algorithms

| Algorithm      | Type      | Best case                                            | Worst case                  | Extra memory  | Needs sorted data? | Stable? |
|----------------|-----------|------------------------------------------------------|-----------------------------|---------------|--------------------|---------|
| Bubble Sort    | Sorting   | $O(N^2)$ as written; $O(N)$ with an early-stop check | $O(N^2)$                    | None — $O(1)$ | —                  | Yes     |
| Selection Sort | Sorting   | $O(N^2)$                                             | $O(N^2)$                    | None — $O(1)$ | —                  | No      |
| Linear Search  | Searching | $O(1)$ (first item)                                  | $O(N)$ (last item / absent) | None — $O(1)$ | No                 | —       |
| Binary Search  | Searching | $O(1)$ (item at first MID)                           | $O(\log N)$                 | None — $O(1)$ | Yes                | —       |

| Point                | Bubble Sort                   | Selection Sort                            |
|----------------------|-------------------------------|-------------------------------------------|
| What is compared     | Adjacent elements (j and j+1) | Fixed position K with every later element |
| Inner loop starts at | 0                             | outer + 1                                 |
| After each pass      | Largest moves to the end      | Smallest moves to the front               |
| Swaps                | Many                          | Fewer                                     |
| Stable               | Yes                           | No                                        |

| Point                 | Linear Search                             | Binary Search                           |
|-----------------------|-------------------------------------------|-----------------------------------------|
| Method                | Checks one by one                         | Splits the list in half each step       |
| Data required         | Sorted or unsorted                        | Sorted only                             |
| Worst case (1M items) | 1,000,000 checks                          | About 20 checks                         |
| Code                  | Very simple                               | More complex, off-by-one risk           |
| Best for              | Small or unsorted lists, one-off searches | Large, sorted lists searched many times |

#### WORKED EXAMPLE

##### Choosing in practice

- **A shopkeeper's list of 15 items, written in random order, searched once:** Linear Search — the list is unsorted and tiny; sorting first would waste time.
- **NADRA looking up a CNIC number among millions of sorted records, thousands of times a day:** Binary Search — data is sorted, huge, and searched repeatedly.
- **A teacher arranging 40 students' marks into a merit list:** a sorting algorithm; for 40 items even Bubble or Selection Sort is acceptable.
- **An unsorted list of 1 million records that will be searched thousands of times:** sort it once, then use Binary Search for every search.

#### REMEMBER IT

Two keywords decide most exam answers: "**sorted?**" and "**how big?**". Unsorted or small → Linear. Sorted and big → Binary.

#### COMMON MISTAKE

"Binary Search is always better" is wrong. On unsorted data it does not work at all, and for a single search on a small unsorted list, sorting first costs more than a simple Linear Search.

#### اردو میں سمجھیں

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔ غیر ترتیب شدہ ڈیٹا میں تلاش کے لیے لینیئر سرچ، ترتیب شدہ ڈیٹا میں تلاش کے لیے بائنری سرچ، اور ڈیٹا کو ترتیب دینے کے لیے سارٹنگ الگورتھم استعمال کریں۔

#### CHECK YOURSELF

1. Which search would you use on an unsorted list of phone contacts?
2. What is the worst-case time complexity of Binary Search?
3. Which sort has the same time complexity in its best and worst case?

Answers: 1. Linear Search. 2.  $O(\log N)$ . 3. Selection Sort ( $O(N^2)$  in both).

Next lecture: Chapter review (in the chapter lecture PDF)